Unit 9 CNN Model Activity, changing input images

I ran the provided CNN object recognition notebook and tested predictions on different test images by changing the input index in x_test. The notebook's original example uses k = 16, so I first ran that baseline case, then switched the input to other indices within 1 to 15. For each case, I generated a predicted label using model.predict and compared it to the ground truth label from y_test.

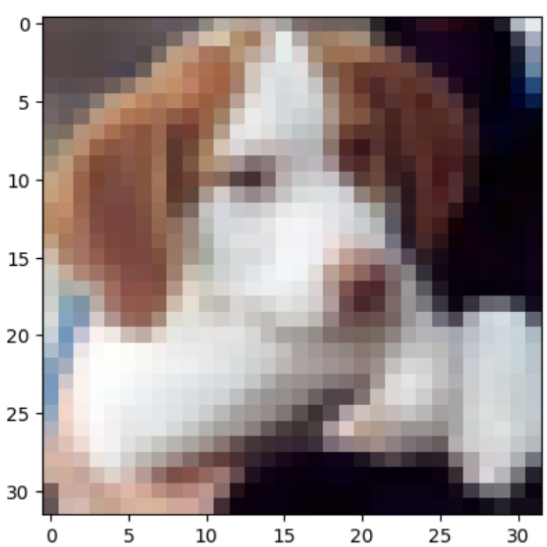
Important Note: CNN based vision systems can cause real harm when used in high impact contexts such as facial recognition and surveillance. A key issue is bias, where error rates differ across demographic groups, often due to imbalanced training data and deployment context. This is not just technical error, because mistakes can lead to unfair outcomes for individuals. Responsible use needs transparency about limitations, testing beyond overall accuracy, and clear accountability and oversight in deployment.

Results summary:

- k = 16 (original example)
    - Predicted: dog
    - True: dog
    - Correct: Yes

```
[113]: plt.imshow(x_test[16])
```

```
[113]: <matplotlib.image.AxesImage at 0x20c8efc29d0>
```




```
[114]: my_image = x_test[16]
```

```
[115]: import numpy as np

       pred = model.predict(my_image.reshape(1,32,32,3))
       pred_class = np.argmax(pred, axis=1)[0]
       pred_label = LABEL_NAMES[pred_class]

       pred_label
```

```
1/1 [==============================] - 0s 14ms/step
[115]: 'dog'
```

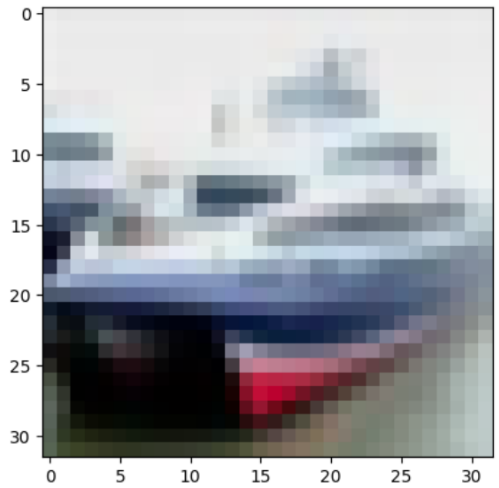
```
[116]: LABEL_NAMES[y_test[16][0]]
```

```
[116]: 'dog'
```

- k = 1 (changed input)
    - Predicted: ship
    - True: ship
    - Correct: Yes

```
[ ]: plt.imshow(x_test[1]) #tested image
```

```
[49]: <matplotlib.image.AxesImage at 0x2a1a4a459d0>
```

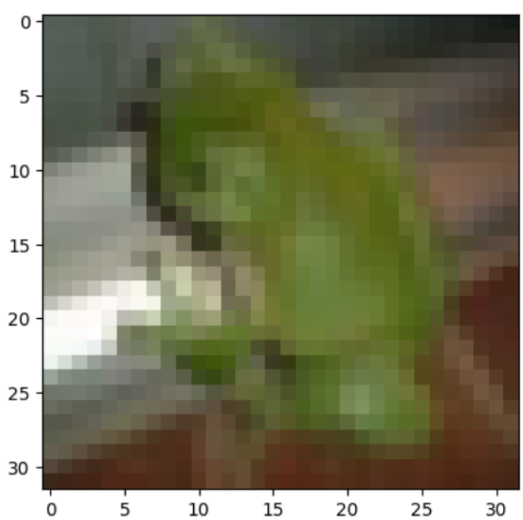



```
[ ]: my_image = x_test[1] #tested image
     import numpy as np

     pred = model.predict(my_image.reshape(1,32,32,3))
     pred_class = np.argmax(pred, axis=1)[0]
     pred_label = LABEL_NAMES[pred_class]

     pred_label

     LABEL_NAMES[y_test[1][0]]

     1/1 [==============================] - 0s 10ms/step
[54]: 'ship'
```

- k = 7 (changed input)
    - Predicted: frog
    - True: frog
    - Correct: Yes

```
[51]: plt.imshow(x_test[7]) #tested image
```

```
[51]: <matplotlib.image.AxesImage at 0x2a1c1ac5b50>
```




```
[55]: my_image = x_test[7] #tested image
      import numpy as np

      pred = model.predict(my_image.reshape(1,32,32,3))
      pred_class = np.argmax(pred, axis=1)[0]
      pred_label = LABEL_NAMES[pred_class]

      pred_label

      LABEL_NAMES[y_test[7][0]]

      1/1 [==============================] - 0s 11ms/step
[55]: 'frog'
```

This demonstrated how the trained CNN behaves on different individual inputs rather than only the default example, and it provided a simple way to validate whether predictions align with the true labels.